

BGlabs2

姓名	杨力泉
学号	231220069
日期	2026 年 4 月 14 日

一、本地实验环境

- 操作系统：Ubuntu 22.04 (WSL2)
- JDK 版本：OpenJDK 1.8.0_281
- Hadoop 版本：3.2.1
- 硬件配置：vCPU2核，内存4GB，硬盘30G

二、任务一：无停用词倒排索引

2.1 设计思路

- 倒排索引要求输出：词语 → 总词频 → 文档1:词频 → 文档2:词频 ...，并按总词频降序排列。由于一个 MapReduce 作业默认按 Key 升序排序，无法直接满足要求，因此采用两个 MapReduce 作业串联：

Job1（词频统计,源代码如下）

- Map：读取每个评论文件，提取文档名（文件名）和单词，输出格式为，key:(单词),value:(文档名:1)。

```
protected void map(LongWritable key, Text value, Context ctx) throws IOException, InterruptedException {
    String line = value.toString();
    String[] parts = line.split(",", 2);
    if (parts.length < 2) return;
    String[] words = parts[1].split("\\s+");
    for (String w : words) {
        if (w.trim().isEmpty()) continue;
        ctx.write(new Text(w), new Text(filename + ":1"));
    }
}
```
- Reduce：对同一单词的所有记录按文档名聚合，统计每个文档的词频并计算总词频，输出格式为,key:(单词), value：（总词频\t文档1:词频\t文档2:词频...）。

```
@Override
protected void reduce(Text key, Iterable<Text> values, Context ctx) throws IOException, InterruptedException {
    Map<String, Integer> map = new HashMap<>();
    int total = 0;
    for (Text val : values) {
        String[] kv = val.toString().split(":");
        String doc = kv[0];
        int cnt = Integer.parseInt(kv[1]);
        map.put(doc, map.getOrDefault(doc, 0) + cnt);
        total += cnt;
    }
    StringBuilder sb = new StringBuilder();
    sb.append(total);
    for (Map.Entry<String, Integer> e : map.entrySet()) {
        sb.append("\t").append(e.getKey()).append(":").append(e.getValue());
    }
    ctx.write(key, new Text(sb.toString()));
}
```

Job2（排序，非核心，源代码未贴出）

- Map：读取 Job1 的输出，将总词频作为 Key，整行记录作为 Value，输出 (总词频, 原始行)。
- Reduce：利用 Hadoop 的降序比较器，直接输出 Value 即可得到按总词频降序排列的结果。

2.2 输出结果与WebUI 执行报告

1	的	91667	video4.csv:21691	video5.csv:1082	video3.csv:7505	video1.csv:17630	video2.csv:43759
2	我	34011	video4.csv:7853	video5.csv:614	video3.csv:4127	video1.csv:4997	video2.csv:16420
3	是	29619	video4.csv:6750	video5.csv:294	video3.csv:2446	video1.csv:7602	video2.csv:12527
4	了	28430	video4.csv:6930	video5.csv:1124	video3.csv:2951	video1.csv:5505	video2.csv:11920
5	你	14904	video4.csv:2259	video5.csv:312	video3.csv:1723	video1.csv:2593	video2.csv:8017
6	不	12596	video4.csv:2656	video5.csv:191	video3.csv:1201	video1.csv:2408	video2.csv:6140
7	就	11119	video4.csv:2644	video5.csv:165	video3.csv:827	video1.csv:2407	video2.csv:5076
8	也	11092	video4.csv:3136	video5.csv:210	video3.csv:917	video1.csv:2262	video2.csv:4567
9	读书	11031	video4.csv:3	video1.csv:36	video2.csv:10992		
10	在	10951	video4.csv:2806	video5.csv:196	video3.csv:1018	video1.csv:2274	video2.csv:4657
11	有	10566	video4.csv:2190	video5.csv:125	video3.csv:1052	video1.csv:2066	video2.csv:5133
12	:	10432	video4.csv:2584	video5.csv:266	video3.csv:1044	video1.csv:1807	video2.csv:4731
13	人	10429	video4.csv:2057	video5.csv:108	video3.csv:960	video1.csv:1821	video2.csv:5483
14	回复	10074	video4.csv:2571	video5.csv:293	video3.csv:979	video1.csv:1872	video2.csv:4359
15	都	10000	video4.csv:2279	video5.csv:169	video3.csv:1081	video1.csv:2196	video2.csv:4275
16	自己	9775	video4.csv:1424	video5.csv:46	video3.csv:1040	video1.csv:1023	video2.csv:6242
17	说	8199	video4.csv:1950	video5.csv:68	video3.csv:452	video1.csv:2283	video2.csv:3446
18	看	8031	video4.csv:3094	video5.csv:122	video3.csv:515	video1.csv:1406	video2.csv:2894
19	和	7267	video4.csv:1988	video5.csv:79	video3.csv:555	video1.csv:1343	video2.csv:3302
20	一个	6172	video4.csv:1652	video5.csv:68	video3.csv:616	video1.csv:1174	video2.csv:2662

ID	User	Name	Application Type	Queue	Application Priority	StartTime	LaunchTime	FinishTime	State	FinalS
application_1774860907544_0509	231220069a	InvertedIndex_Job2_Sort	MAPREDUCE	2025class02	0	Tue Apr 14 00:55:28 +0800 2026	Tue Apr 14 00:55:28 +0800 2026	Tue Apr 14 00:55:44 +0800 2026	FINISHED	SUCCE
application_1774860907544_0508	231220069a	InvertedIndex_Job1	MAPREDUCE	2025class02	0	Tue Apr 14 00:55:10 +0800 2026	Tue Apr 14 00:55:10 +0800 2026	Tue Apr 14 00:55:26 +0800 2026	FINISHED	SUCCE

HDFS 输出路径：/user/231220069a/output_task1/

三、任务二：带停用词过滤的倒排索引

3.1 设计思路

- 在任务一的基础上增加停用词过滤。停用词表 hit_stopwords.txt 通过 HDFS 路径直接读取，在 Mapper 的 setup() 中加载为 Set。Map 阶段对每个单词判断：若在停用词集合中则跳过，否则正常输出。其余流程（包括排序作业）与任务一完全相同。即p2的除了mapper部分与p1的完全一致（代码复用）。

Key / Value 类型

- 与任务一完全相同，仅 Mapper 增加了停用词过滤逻辑。

```
private String filename;
private Set<String> stopWords;
@Override
protected void setup(Context ctx) throws IOException, InterruptedException {
    filename = ((FileSplit) ctx.getInputSplit()).getPath().getName();
    String stopPath = ctx.getConfiguration().get("stopword.path");
    try {
        stopWords = StopWordLoader.load(ctx.getConfiguration(), stopPath);
    } catch (Exception e) {}
    stopWords = new HashSet<>();
}

@Override
protected void map(LongWritable key, Text value, Context ctx) throws IOException, InterruptedException {
    String line = value.toString();
    String[] parts = line.split(" ", 2);
    if (parts.length < 2) return;
    String[] words = parts[1].split("\\s+");
    for (String w : words) {
        w = w.trim();
        if (w.isEmpty() || stopWords.contains(w)) continue;
        ctx.write(new Text(w), new Text(filename + ":1"));
    }
}
```

3.2 输出结果与WebUI 执行报告

1	的	91667	video4.csv:21691	video5.csv:1082	video3.csv:7505	video1.csv:17630	video2.csv:43759
2	我	34011	video4.csv:7853	video5.csv:614	video3.csv:4127	video1.csv:4997	video2.csv:16420
3	是	29619	video4.csv:6750	video5.csv:294	video3.csv:2446	video1.csv:7602	video2.csv:12527
4	了	28430	video4.csv:6930	video5.csv:1124	video3.csv:2951	video1.csv:5505	video2.csv:11920
5	你	14904	video4.csv:2259	video5.csv:312	video3.csv:1723	video1.csv:2593	video2.csv:8017
6	不	12596	video4.csv:2656	video5.csv:191	video3.csv:1201	video1.csv:2408	video2.csv:6140
7	就	11119	video4.csv:2644	video5.csv:165	video3.csv:827	video1.csv:2407	video2.csv:5076
8	也	11092	video4.csv:3136	video5.csv:210	video3.csv:917	video1.csv:2262	video2.csv:4567
9	读书	11031	video4.csv:3	video1.csv:36	video2.csv:10992		
10	在	10951	video4.csv:2806	video5.csv:196	video3.csv:1018	video1.csv:2274	video2.csv:4657
11	有	10566	video4.csv:2190	video5.csv:125	video3.csv:1052	video1.csv:2066	video2.csv:5133
12	人	10429	video4.csv:2057	video5.csv:108	video3.csv:960	video1.csv:1821	video2.csv:5483
13	回复	10074	video4.csv:2571	video5.csv:293	video3.csv:979	video1.csv:1872	video2.csv:4359
14	都	10000	video4.csv:2279	video5.csv:169	video3.csv:1081	video1.csv:2196	video2.csv:4275
15	自己	9775	video4.csv:1424	video5.csv:46	video3.csv:1040	video1.csv:1023	video2.csv:6242
16	说	8199	video4.csv:1950	video5.csv:68	video3.csv:452	video1.csv:2283	video2.csv:3446
17	看	8031	video4.csv:3094	video5.csv:122	video3.csv:515	video1.csv:1406	video2.csv:2894
18	和	7267	video4.csv:1988	video5.csv:79	video3.csv:555	video1.csv:1343	video2.csv:3302
19	一个	6172	video4.csv:1652	video5.csv:68	video3.csv:616	video1.csv:1174	video2.csv:2662
20	很	6117	video4.csv:2151	video5.csv:60	video3.csv:545	video1.csv:1079	video2.csv:2282

application_1774860907544_0511	231220069a	Task2_Sort	MAPREDUCE	2025class02	0	Tue Apr 14 01:01:08 +0800 2026	Tue Apr 14 01:01:08 +0800 2026	Tue Apr 14 01:01:24 +0800 2026	FINISHED	SUCCEEDED	N/A
application_1774860907544_0510	231220069a	Task2_WithStopWords	MAPREDUCE	2025class02	0	Tue Apr 14 01:00:50 +0800 2026	Tue Apr 14 01:00:50 +0800 2026	Tue Apr 14 01:01:06 +0800 2026	FINISHED	SUCCEEDED	N/A

HDFS 输出路径：/user/231220069a/output_task2/

四、任务三（选做）：中文小说分词 + 倒排索引

4.1 设计思路

- 任务三分两步：分词+任务一

中文分词

- 输入：task3/ 下的多个 .txt 小说文件（未分词）。使用 HanLP 对每行进行分词，过滤掉长度<2的字符和纯数字。输出格式模仿任务一的输入：虚拟ID, 分词结果（虚拟ID = 文件名_行号），以便后续直接复用

任务一的代码。

```
public class Task3SegmentMapper extends Mapper<LongWritable, Text, Text, NullWritable> {
    private String filename;

    @Override
    protected void setup(Context ctx) {
        filename = ((FileSplit) ctx.getInputSplit()).getPath().getName();
    }

    @Override
    protected void map(LongWritable key, Text value, Context ctx) throws IOException {
        String line = value.toString();
        if (line.trim().isEmpty()) return;
        List<Term> terms = HanLP.segment(line);
        StringBuilder sb = new StringBuilder();
        for (Term t : terms) {
            String word = t.word;
            if (word.length() < 2) continue;
            if (word.matches("\\d+")) continue;
            sb.append(word).append(" ");
        }
        if (sb.length() > 0) {
            String virtualId = filename + "_" + key.get();
            String outLine = virtualId + "," + sb.toString().trim();
            ctx.write(new Text(outLine), NullWritable.get());
        }
    }
}
```

倒排索引（复用task1）

- 再直接调用任务一的 Driver，输入为分词输出目录，输出为最终倒排索引结果。

3.2 输出结果与WebUI 执行报告

一个	5679	part-m-00001:1067	part-m-00023:60	part-m-00022:57	part-m-00000:1399	part-m-00021:7
我们	3709	part-m-00001:679	part-m-00023:10	part-m-00000:480	part-m-00022:23	part-m-00021:3
没有	3050	part-m-00001:655	part-m-00023:23	part-m-00000:908	part-m-00022:14	part-m-00021:1
他们	3026	part-m-00023:9	part-m-00001:648	part-m-00000:740	part-m-00022:25	part-m-00021:3
这个	2318	part-m-00001:497	part-m-00023:14	part-m-00000:568	part-m-00022:17	part-m-00021:1
自己	2205	part-m-00001:530	part-m-00023:11	part-m-00000:557	part-m-00022:5	part-m-00021:1
现在	2185	part-m-00001:545	part-m-00023:12	part-m-00000:507	part-m-00022:21	part-m-00021:2
看到	1994	part-m-00001:342	part-m-00023:19	part-m-00000:492	part-m-00022:11	part-m-00021:2
什么	1928	part-m-00023:15	part-m-00001:442	part-m-00022:11	part-m-00000:399	part-m-00021:1
世界	1921	part-m-00001:329	part-m-00023:8	part-m-00000:671	part-m-00022:7	part-m-00021:1
已经	1761	part-m-00001:523	part-m-00023:11	part-m-00000:547	part-m-00022:3	part-m-00021:5
知道	1747	part-m-00023:12	part-m-00001:435	part-m-00000:416	part-m-00022:13	part-m-00021:1
.....	1612	part-m-00023:32	part-m-00001:432	part-m-00000:32	part-m-00022:54	part-m-00021:39
地球	1593	part-m-00001:253	part-m-00023:1	part-m-00000:408	part-m-00021:44	part-m-00020:6
可能	1571	part-m-00001:392	part-m-00023:3	part-m-00000:489	part-m-00022:13	part-m-00021:1
人类	1518	part-m-00001:396	part-m-00000:455	part-m-00021:45	part-m-00020:14	part-m-00009:2
中的	1400	part-m-00001:291	part-m-00023:9	part-m-00022:12	part-m-00000:380	part-m-00021:1
你们	1384	part-m-00001:328	part-m-00023:1	part-m-00000:215	part-m-00022:7	part-m-00021:2
太空	1353	part-m-00001:392	part-m-00000:650	part-m-00021:3	part-m-00020:16	part-m-00008:8
宇宙	1335	part-m-00023:2	part-m-00001:174	part-m-00000:450	part-m-00021:9	part-m-00020:2

ID	User	Name	Application Type	Queue	Application Priority	StartTime	LaunchTime	FinishTime	State	FinalStatus	Running Container
application_1774860907544_0514	231220069a	InvertedIndex_Job2_Sort	MAPREDUCE	2025class02	0	Tue Apr 14 01:04:23 +0800 2026	Tue Apr 14 01:04:23 +0800 2026	Tue Apr 14 01:04:39 +0800 2026	FINISHED	SUCCEEDED	1
application_1774860907544_0513	231220069a	InvertedIndex_Job1	MAPREDUCE	2025class02	0	Tue Apr 14 01:04:05 +0800 2026	Tue Apr 14 01:04:05 +0800 2026	Tue Apr 14 01:04:21 +0800 2026	FINISHED	SUCCEEDED	N/A
application_1774860907544_0512	231220069a	ChineseSegment	MAPREDUCE	2025class02	0	Tue Apr 14 01:03:14 +0800 2026	Tue Apr 14 01:03:14 +0800 2026	Tue Apr 14 01:03:26 +0800 2026	FINISHED	SUCCEEDED	N/A

HDFS 输出路径：**/user/231220069a/output_segmented/**(中间结果)

HDFS 输出路径：**/user/231220069a/output_task3_result/**

五、附件说明

压缩包 **231220069_杨力泉_BG_2.zip** 包含：

- 源代码：所有 .java 文件（位于 src/ 目录）
- pom.xml
- JAR 包：inverted-index-1.0-jar-with-dependencies.jar
- 执行方式说明：README（含各任务完整命令）
- 实验结果（data_ans目录）
- 实验报告：本文件